

# Avoid Exposing Concurrency: Hide It Inside Synchronous Methods

**You have a mass of existing code and want to add concurrency. Where do you start?**

**By Herb Sutter**

*Herb Sutter is a bestselling author and consultant on software development topics, and a software architect at Microsoft. He can be contacted at [www.gotw.ca](http://www.gotw.ca).*

---

Let's say you need to migrate existing code to take advantage of concurrent execution or scale on parallel hardware. In that case, you'll probably find yourself in one of these two common situations, which are actually more similar than different:

- Migrating an application: You're an application developer, and you want to migrate your existing synchronous application to be able to benefit from concurrency.
- Migrating a library or framework: You're a developer on a team that produces a library or framework used by other teams or external customers, and you want to let the library take advantage of concurrency on behalf of the application without requiring application code rewrites.

You have a mountain of opportunities and obstacles before you. Where do you start?

This column is about the most basic strategy to have in our toolchest. It may seem so simple that it goes without saying, but there are a few important details to keep in mind to do it correctly.

## **Adding Concurrency: Target It, Localize It, Hide It**

Perhaps the most effective way to deliver concurrency is to hide it inside "synchronous" APIs. The key is to inject concurrency in a targeted, localized way inside key functions or methods, and let the application code that calls those methods stay happily sequential and blissfully unaware of the concurrency. We want keep from exposing concurrency throughout the entire application, because the existing code isn't designed with concurrency and nondeterminism in mind. The less code has to be upgraded to know about concurrency, the better.

First, identify the places where we can benefit from concurrency. Here are two major kinds of situations to look for:

- High-latency operations that can benefit from concurrent waiting. Here we want to look for places where our lower-level or library code is doing synchronous work when the caller requests it. If that work could be launched actively sooner (even if sometimes speculatively), then the waiting can overlap with the caller's execution, so that when the caller finally does ask for the result some or all of the work or waiting is already complete.
- Compute-intensive operations that can benefit from parallel execution. Here, we want to look for "hot spot" functions or methods where your application or library code is spending significant parts of computational effort. Focus on the ones that are amenable to parallelization because we can readily break the work into chunks that can be performed in parallel. Typically, we're looking for natural parallelism that we can exploit in the algorithm (e.g., recursive divide-and-conquer where nonoverlapping subranges can be processed in parallel) or in the data structures (e.g., nonoverlapping subtrees or subgraphs or hash table buckets can be processed in parallel). These are our initial targets.

Of course, there's overlap between these two categories, as we'll see in the following examples. Second, once we've found that low-hanging fruit, add concurrency in those functions internally. Note that word "internally" is essential -- we don't want to expose the concurrency in the API we present to the caller. Ideally, all concurrency is bounded within the scope of the function call, so that the caller calls our function and we launch the concurrency and join with it before returning, and the function just appears to execute faster. Next best, concurrency is not bounded but is managed automatically, so that the caller calls our function and we launch the concurrency and return with something still running under the covers, and the caller doesn't need to know because in some subsequent call into our code (or upon a timeout, or some other automated way) we join with the concurrent work.

With the concurrency thus compartmentalized, the calling code doesn't have to know about the parallel work. As far as the caller is concerned, our library method just happens to execute better or faster than before.

### Example: An Internally Parallelized Algorithm

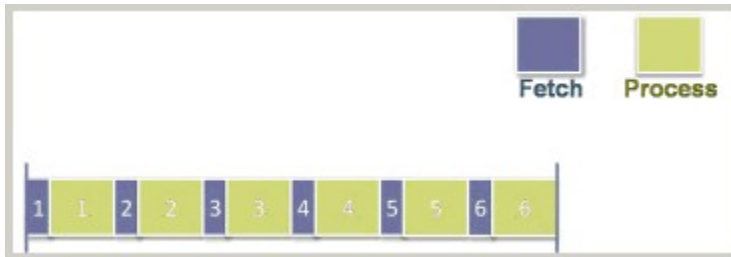
The first example is simple and obvious, but shows an important way to deliver scalable parallelism in a tactical way for an existing application. Consider the following sequential implementation of quicksort, simplified to its essentials:

```
// Example 1(a): Sequential quicksort
void Quicksort( Iter first, Iter last ) {
    // If the range is small, another sort is faster.
    if( distance( first, last ) < limit ) {
        OtherSort( first, last );
        return;
    }
    // 1. Pick a pivot and partition the data.
    Iter pivot = Partition( first, last );
    // 2. Recurse to sort the subranges.
    Quicksort( first, pivot );
    Quicksort( pivot, last );
}
```

This is a traditional divide-and-conquer recursive algorithm, performed sequentially. If we're trying to parallelize our application, this is just the sort (sorry) of function we're looking for: A compute-intensive function that contains natural parallelism, in this case in the algorithm. The subrange sort operations work on nonoverlapping subsets of the data, and their processing is probably fully independent as long as sorting the left subrange doesn't cause other side effects that could affect sorting the right subrange, and vice versa.

This turns out to be a perfectly simple case to illustrate the point of this article, because the natural way to parallelize this code inherently hides the parallelism from the caller (assuming we get the mechanical details right; more about avoiding mistakes and pitfalls in another article):

```
// Example 1(b): Parallelized quicksort
void Quicksort( Iter first, Iter last ) {
    // If the range is small, synchronous sort is faster.
    if( distance( first, last ) < limit ) {
        OtherSort( first, last );
        return;
    }
    // 1. Pick a pivot and partition the data synchronously.
    Iter pivot = Partition( first, last );
    // 2. Recurse to sort the subranges in parallel.
    future fut = pool.run( [=]{ Quicksort( first, pivot ); } );
    Quicksort( pivot, last );
    fut.join();
}
```



**Figure 1: Synchronous fetch-process**

Note that the code may look slightly different in your environment, but we're simply saying to execute the two subrange sorts in parallel:

- The syntax **pool.run(x)** denotes creating a task **x** to run in parallel, which is common when using thread pools for example.
- The syntax **[=]{ f(); }** is C++ lambda function syntax, a convenient syntactic sugar for writing a function object to be executed elsewhere. If you're using C#, the syntax looks like **()=>{ f(); }**. If you're using Java, just create a runnable object by hand.

This function may spawn a lot of parallel work recursively, but everything is joined before we return to the caller. The function just happens to run faster when run on machine that has more cores. So sequential code that calls this library function also gets to reap the benefits of leveraging whatever hardware parallelism there happens to be on a given end user's machine, at least for this function, without having to know about the parallel work being done on its behalf.

### Example: Optimistic Iteration

Now consider the following loop that uses an enumerator/iterator style to visit each item in a collection:

```
// Example 2, common calling code
void f( WidgetCollection wc ) {
    using( i = wc.GetEnumerator() ) {
        while( i.MoveNext() ) { // fetch next element
            // some work // and process it
            DoSomethingWith( i.Current );
            // more work
        }
    }
}
```

Imagine the enumerator is implemented something like this:

```
// Example 2(a), synchronous enumerator (sketch)
class WidgetEnumerator {
    //
    // Construct
    WidgetEnumerator() {
```

```

    // set up any necessary resources
}
// Destroy (C++ destructor, Java/C# disposer)
void Dispose() {
    // release any resources we got
}
bool MoveNext() {
    if( AtEnd() ) {
        return false;
    }
    // note: this may be an expensive operation
    NavigateToNextItem();
    return true;
}
Widget Current() {
    lock( myWidgetQueue ) {
        return myWidgetQueue.First;
    }
}

```

Each time the caller asks for the next element, we go look for it, then return and let the caller process it. So in this sequential version of the code, the fetching and processing work is interleaved, as in Figure 1.

What if **NavigateToNextItem** (and therefore **MoveNext**) is an expensive operation? What if it has to perform an expensive computation, or take another round trip to a disk or database or web service? Then it's quite likely that we can do significantly better even for this simple common iteration pattern by applying concurrency.

Everyone who has implemented or used an enumeration API like this knows that, if the caller is asking for one item, he's likely to soon come back, hat in hand, and ask for the next one too, and then the next one after that. If getting the next element takes nontrivial time for any of a variety of reasons, and we have resources to spare, we may want to be more proactive about generating the results so that we can deliver them faster once the caller actually requests them. And, if we guess wrong, we want a way to throw away the results we didn't need after all. Even if the cost to find and prepare the next item is less than the time for the caller to complete a loop iteration and ask for the next item, as in Figure 1, we can gain a useful speedup by running the enumerator's fetching concurrently with the caller's processing.

So consider performing the iteration concurrently (and speculatively, because if the caller doesn't end up traversing the whole range we may end up throwing some work away):

```

// Example 2(b), concurrent enumerator (sketch)
class WidgetEnumerator {
    //
    // Construct
    WidgetEnumerator() {
        // set up any necessary resources
        // and launch a traversal worker thread
        done = false;
        pleaseStop = false;
    }
}

```

```

myWidgetQueue.Append( /* dummy */ );
myWorker = new thread( () => {
    while( !pleaseStop && !AtEnd() ) {
        // note: this may be an expensive operation
        NavigateToNextItem();
        lock( myWidgetQueue ) {
            myWidgetQueue.Append( /* current item */ );
        }
        newItem.Notify();          // notify consumer
    }
    lock( myWidgetQueue ) {
        myWidgetQueue.Append( /* sentinel */ );
    }
    newItem.Notify();          // notify one last time
} );
}
// Destroy (C++ destructor, Java/C# disposer)
void Dispose() {
    // stop and join the background worker
    pleaseStop = true;
    myWorker.join();
    // release any resources we got
}
bool MoveNext() {
    if( !done ) {
        lock( myWidgetQueue ) {
            myWidgetQueue.PopFirst();
        }
        newItem.Wait();
    }
    lock( myWidgetQueue ) {
        if( myWidgetQueue.First == /* sentinel */ ) {
            done = true;
            return false;
        }
        return true;
    }
}
Widget Current() {
    lock( myWidgetQueue ) {
        return myWidgetQueue.First;
    }
}
}

```

Now we can perform the fetching and processing concurrently, as in Figure 2. We can further improve this code by doing things like bounding the size of the queue, so that in general fewer resources are in use at once and in particular less work is wasted if the caller should happen to throw away the enumerator before reaching the end, but this is enough to sketch the basic strategy.



**Figure 2: Background fetching**

You may have noticed that what we've really done is create a two-stage pipeline, and we'll look at the general strategy of pipelining in more depth in a future article. For now, the key point to notice here is that the caller's code has no idea that it's participating as a pipeline stage, but remains happily oblivious to the concurrency because we've once again managed to hide it under the covers behind a synchronous-looking API.

### **Summary**

The less code has to be upgraded to know about concurrency and nondeterminism, the better. Prefer to keep knowledge of concurrent work compartmentalized: Where possible, hide it behind a "synchronous" API to limit its effect and let callers remain insulated from the concurrency. This is an excellent strategy to reach for first when migrating an existing code base to take advantage of concurrent execution or scale on parallel hardware.

### **Acknowledgment**

Thanks to Terry Crowley for his insightful comments on drafts of this article.